CUNEF UNIVERSIDAD

Exploratory Data Analysis

Data Analysis & Information Extraction

Table of contents

We will learn how to explore the data in a systematic way using the tools we have learn for visualizing and transformation.

The Exploratory Data Analysis is a cycle that consists in…

- Generate questions about the data.
- Answer them visualizing, transforming and modelling.
- Use what we learn of the preceding steps to re-think the questions and find new ones we can answer.

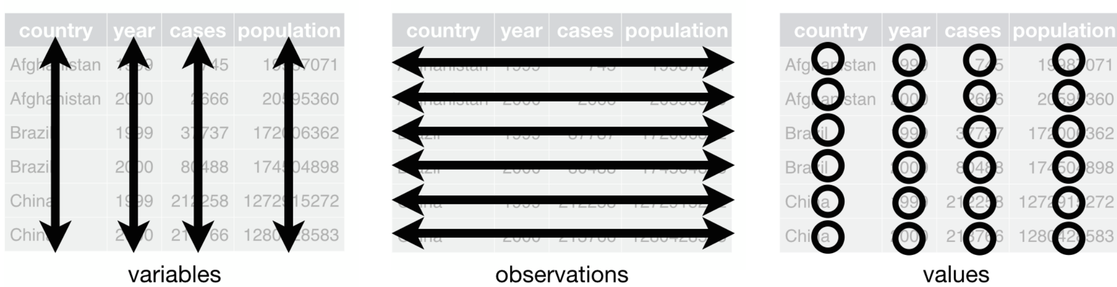
As a creative process, there is no rules for limiting which questions are more useful.

But we will always find interesting two kind of questions…

- *Which kind of variation is present in the variables?*
- *Which kind of co-variation exists between the variables?*

First concepts

- **Variable**: is a quantity, quality, or property that you can measure.
- **Value**: is the state of a variable when you measure it. It may change for measurement to measurement.
- **Observation**: is a set of measurements made under similar conditions. Each observation can contain different values, each value related with their corresponding variable.
- **Tabular data**: a set of values organized in different variables for each observation.



Analyzing Data Variation

Variation is the tendency of the values of a variable to change between each measurement

This variation can happen due to different reasons:

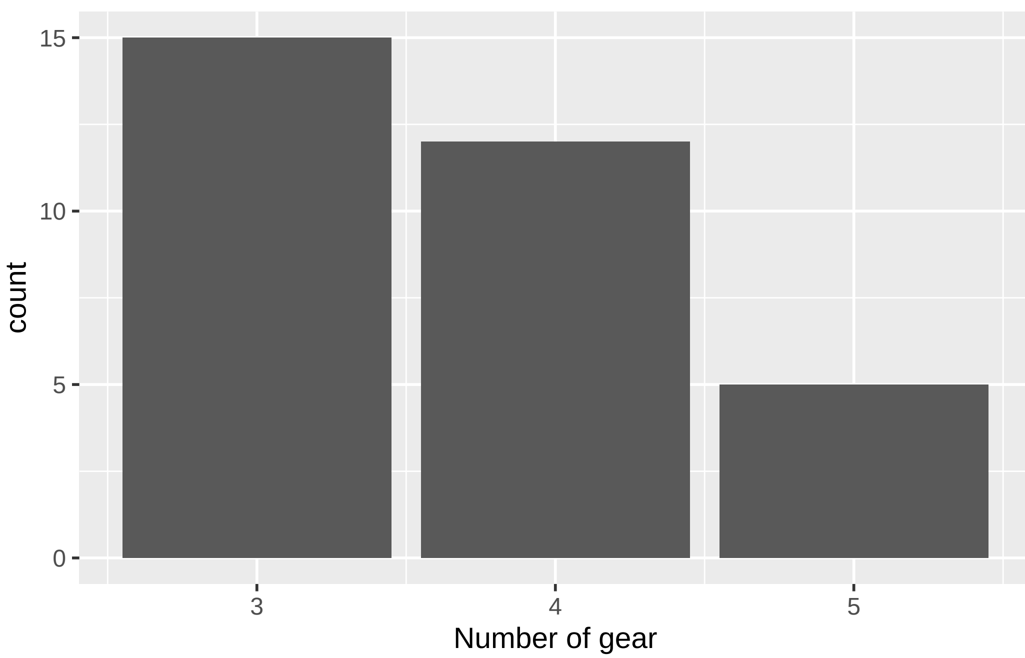
- **Error** when measuring a constant quantity (ligth speed).
- Measuring the same variable in **different subjects**.
- Measuring the same variable for the same subject but **different moment**.
- ...

We can see patterns of the variation, this information can be useful. We can understand these patterns visualizing the **distribution of the different values taken by the variable**.

Variation in categorical variables

Remember that we can visualize the distribution of a categorical variable as follows...

```
library(ggplot2)

# Number of gears for mtcars data set
ggplot(mtcars, aes(x=gear)) +
  geom_bar() +
  labs(x = "Number of gear")
```



Each bar of the last plot shows us the number of observations we can found for every value of the variable. We can obtain the number of observations with this simple code...

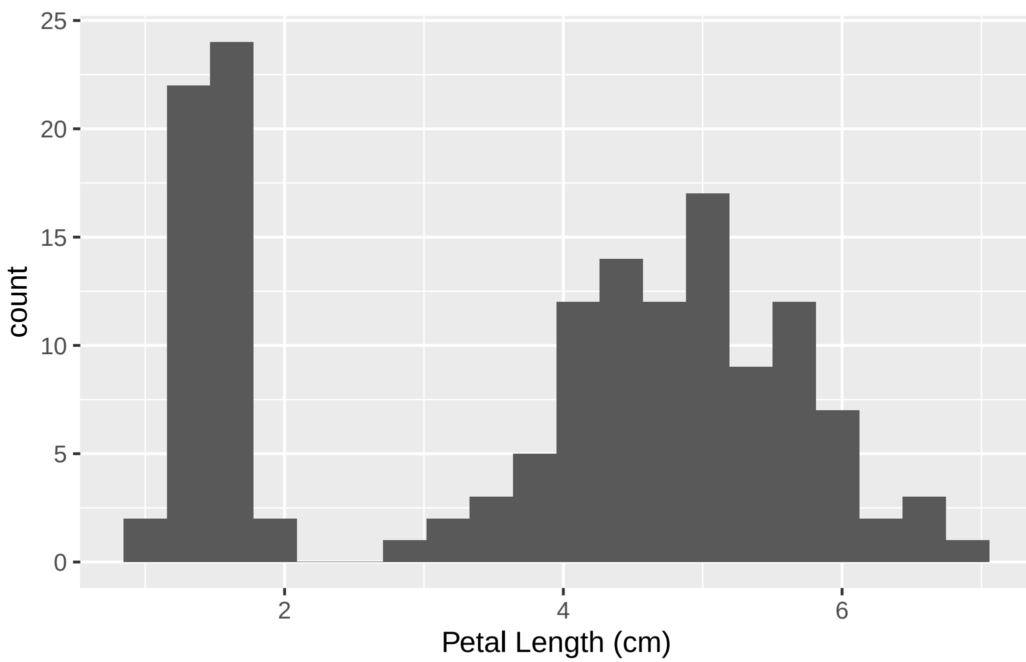
```
library(tidyverse)

mtcars %>%
  group_by(gear) %>%
  summarise(observations = n())
```

```
# A tibble: 3 x 2
   gear observations
  <dbl>        <int>
1     3           15
2     4           12
3     5            5
```

Variation in continuous variables

For visualizing the distribution of continuous variables we can use the histogram...

```
ggplot(data = iris, aes(x = Petal.Length)) +
  geom_histogram(bins = 20) +
  labs(x = "Petal Length (cm)")
```



Analyzing Variation

When analyzing the plots that shows the data variation we need to focus in two things…

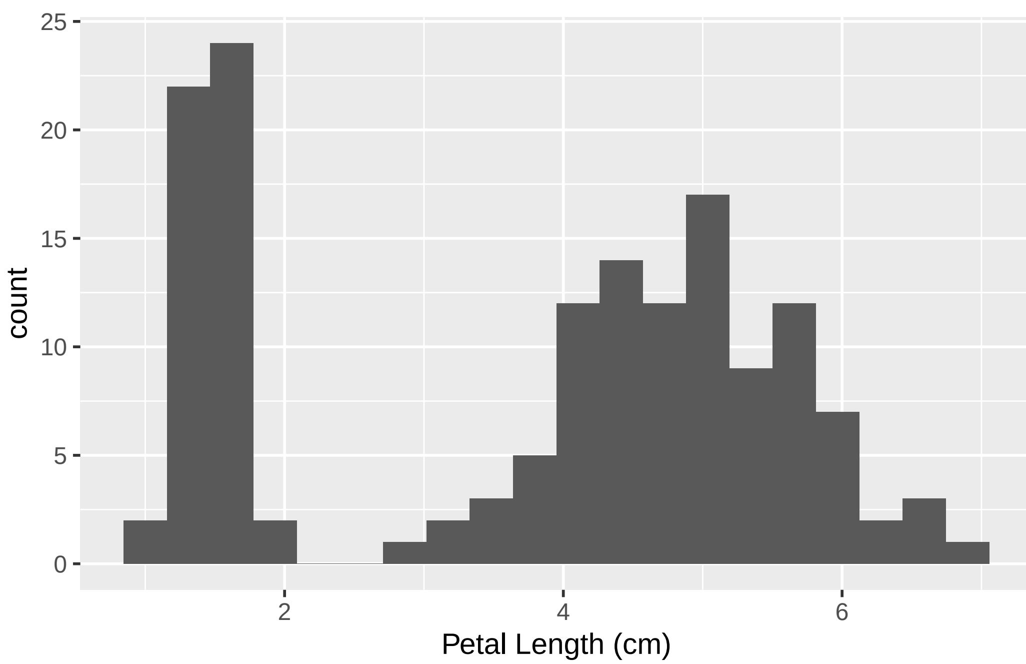
- Normal values
- Anomalies (outliers)

Questions like…

- Most common values?
- Lest common values?
- Strange patterns in the data?

We can see it with an example: *Petal Length Analysis*

```
ggplot(data = iris, aes(x = Petal.Length)) +
  geom_histogram(bins = 20) +
  labs(x = "Petal Length (cm)")
```



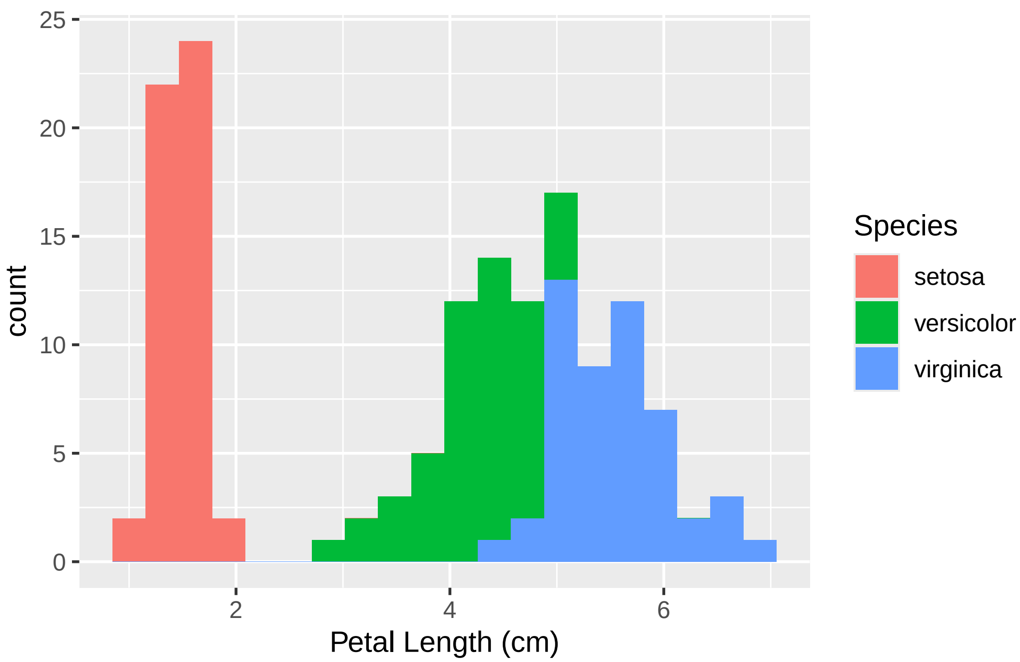
The *clusters* of similar values makes us think there are subgroups in the data. For understanding it, we can ask…

- What have the observations of each group in common?
- What differences are between the observations of each group?
- How can we explain each subgroup?

- There are two groups… one with shorter petal length (<2cm) and other large (>2cm)
- Species variable can explain this separation?

We can analyze multiple histograms in the same plot, using colors for example.

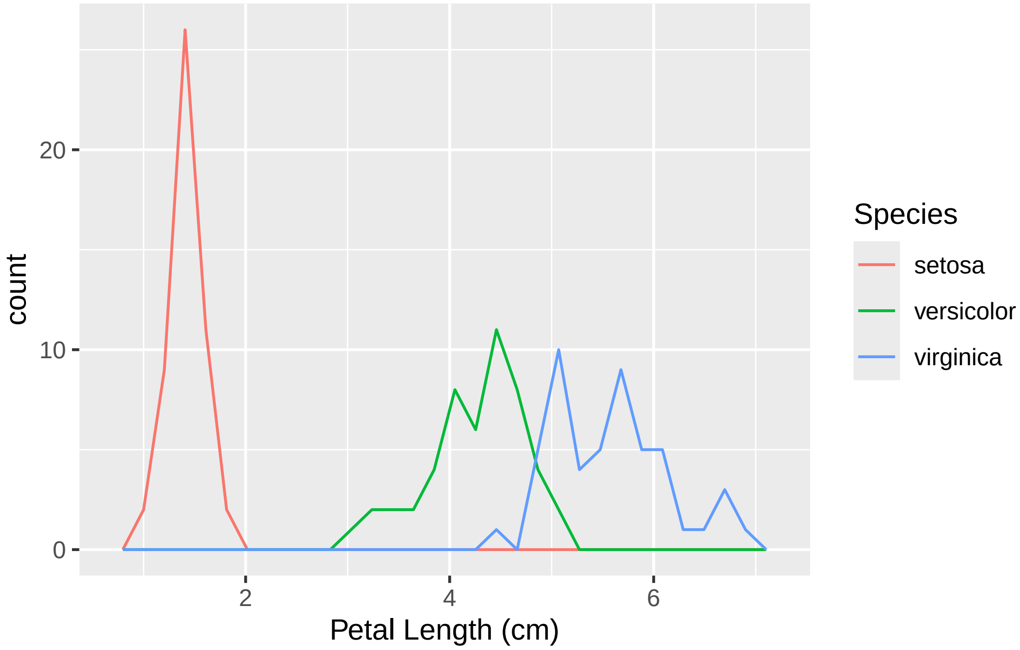
```
ggplot(data = iris,
       aes(x = Petal.Length, fill = Species)) +
  geom_histogram(bins = 20) +
  labs(x = "Petal Length (cm)")
```



Other option, we can use ggplot geometric: `geom_freqpoly()` that works similar to the histogram but showing them with lines.

```
ggplot(data = iris,
       aes(x = Petal.Length, color = Species)) +
  geom_freqpoly() +
  labs(x = "Petal Length (cm)")
```

```
`stat_bin()` using `bins = 30`. Pick better value `binwidth`.
```



Variation: Outliers

Find and analyze **outliers** have the same importance as analyze the normal values we can found.

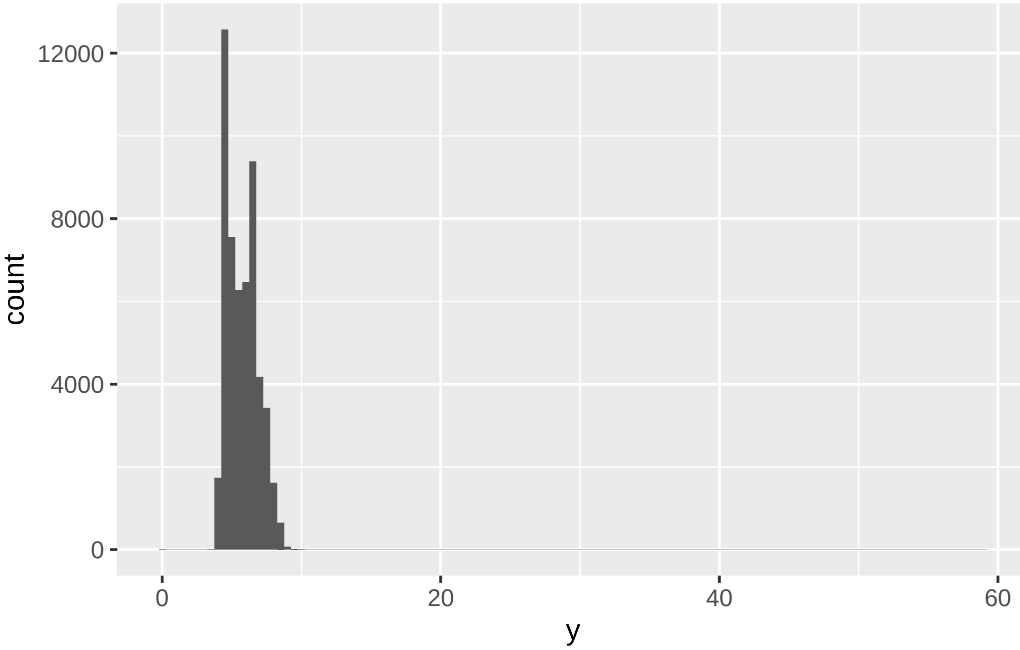
Why we can have outliers…

- Sometimes… measuring error
- Others, further analyze is needed.

We can see it with an example: *Diamonds Width Analysis*

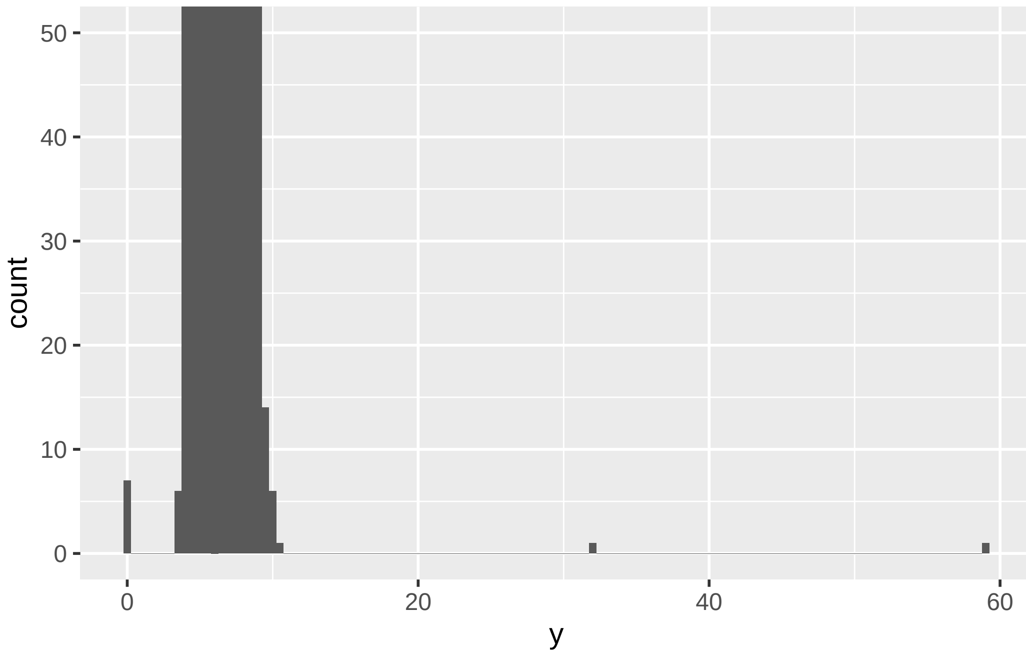
We are going to analyze the distribution of the width of the diamonds in the data set **diamonds** inside R. Apparently all looks good right? (*Hint: Look at x axis?*)

```
ggplot(diamonds, aes(x = y)) +
  geom_histogram(binwidth = 0.5)
```



Looks that there is more space than the one we need to visualizing the data, or probably there is some data we are not able to see, a bit of *zoom* may help…

```
ggplot(diamonds, aes(x=y)) +
  geom_histogram(binwidth = 0.5) +
  coord_cartesian( ylim = c(0,50) )
```



Looks there are some outliers near 0, 30 and 60. We can now go directly to the data and **filter** to extract them..

```
diamonds %>%
  filter( y < 3 | y > 20)
```

```
# A tibble: 9 x 10
  carat cut       color clarity depth table price     x     y     z
  <dbl> <ord>     <ord> <ord>   <dbl> <dbl> <int> <dbl> <dbl> <dbl>
1  1    Very Good H     VS2      63.3    53  5139  0        0     0
2  1.14 Fair      G     VS1      57.5    67  6381  0        0     0
3  2    Premium   H     SI2      58.9    57 12210  8.09  58.9  8.06
4  1.56 Ideal     G     VS2      62.2    54 12800  0        0     0
5  1.2  Premium   D     VVS1     62.1    59 15686  0        0     0
6  2.25 Premium   H     SI2      62.8    59 18034  0        0     0
7  0.51 Ideal     E     VS1      61.8    55  2075  5.15  31.8  5.12
8  0.71 Good      F     SI2      64.1    60  2130  0        0     0
9  0.71 Good      F     SI2      64.1    60  2130  0        0     0
```

Conclusions found after analysis

- Looks there are diamonds with width 0, maybe incorrect data...
- We have detected diamonds with a very big size, but not too expensive...

Actions with outliers

- Try to do the analysis with and without outlers, this will help us identify the impact they have.

Depending on the impact effect…

- Minimum effect and we don't know the origin… replace them as `NA` values.
- Otherwise… eliminate them **justifying** the reason and document / inform their elimination from the analysis.

Droping outliers

When we need to drop some outliers from the data, we have two options…

- Eliminate the whole row (big loss of information)

```
diamonds2 <- diamonds %>%
  filter(y>=3 & y<=20)
```

- Replace them by `NA` (missing value)

```
diamonds2 <- diamonds %>%
  mutate(y = if_else(y<3 | y>20, NA, y))
```

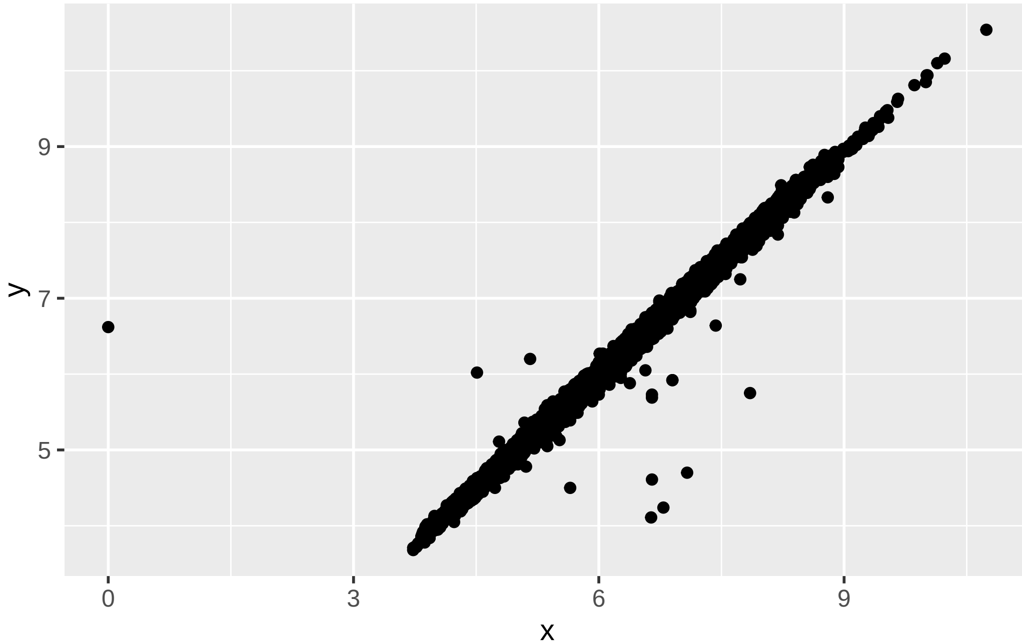
> **if_else**
>
> Conditional function to use inside mutate, first argument the condition, then if TRUE otherwise FALSE in the last argument.

Missing Values

When missing values are present in the data we will see a warning in the plot…

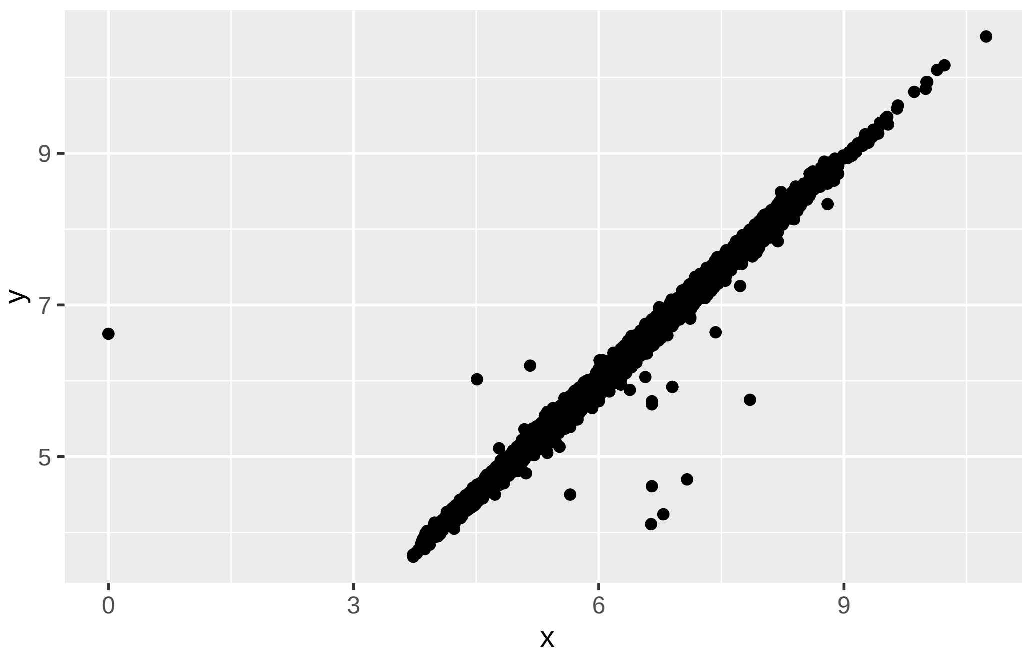
```
ggplot(data=diamonds2,
       mapping = aes(x=x, y=y)) +
  geom_point()
```

```
Warning: Removed 9 rows containing missing values or values outside the scale range
(`geom_point()`).
```



We can quickly eliminate them from the visualization using `na.rm = T` argument…

```
ggplot(data=diamonds2,
       mapping = aes(x=x, y=y)) +
  geom_point(na.rm = T)
```



Analyzing Covariance

> *Covariance is the tendency between two or more values to change simultaneously.*

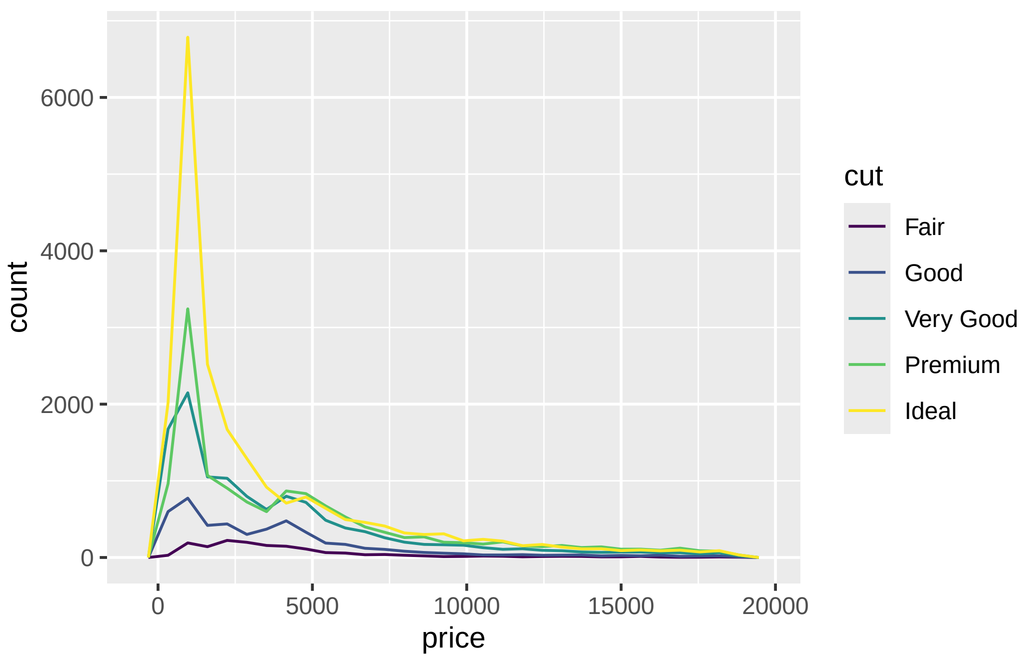
As we have seen, for analyzing covariance we will need to understand the type of variables...

- Categorical - Continuous
- Categorical - Categorical
- Continuous - Continuous

Covariance: Categorical - Continuous

We can use the `geom_freqpoly` geometry to see how the continuous variable relates with the each category.

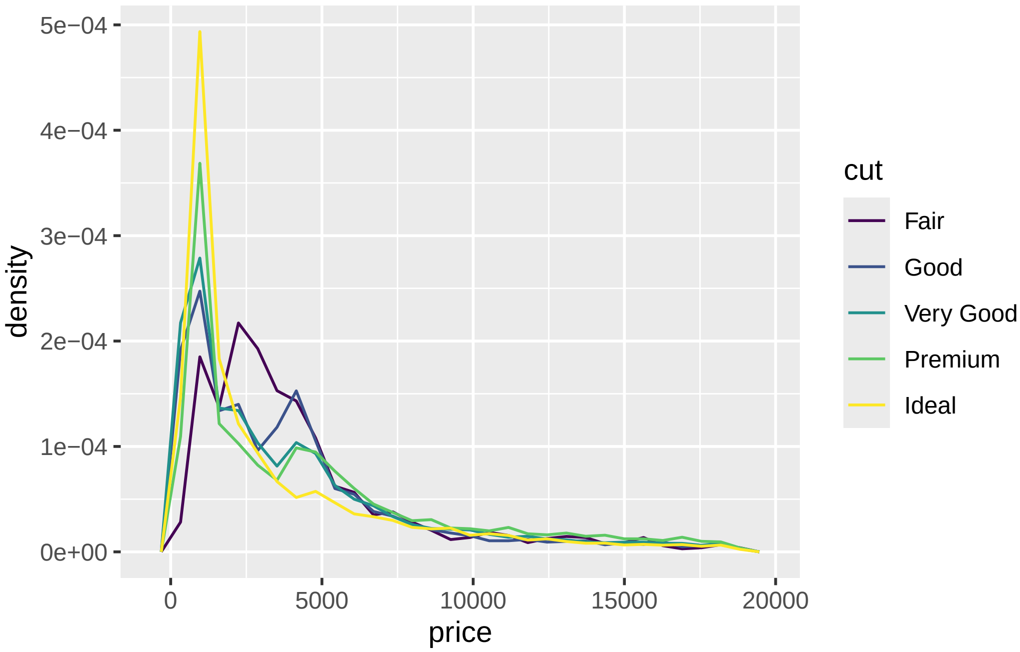
```
ggplot(data = diamonds, aes(x = price, color = cut)) +
  geom_freqpoly()
```



Other option with density plots. `..density..` will paint the density in the y axis, that is the normalized count to make the area of each polygon 1.

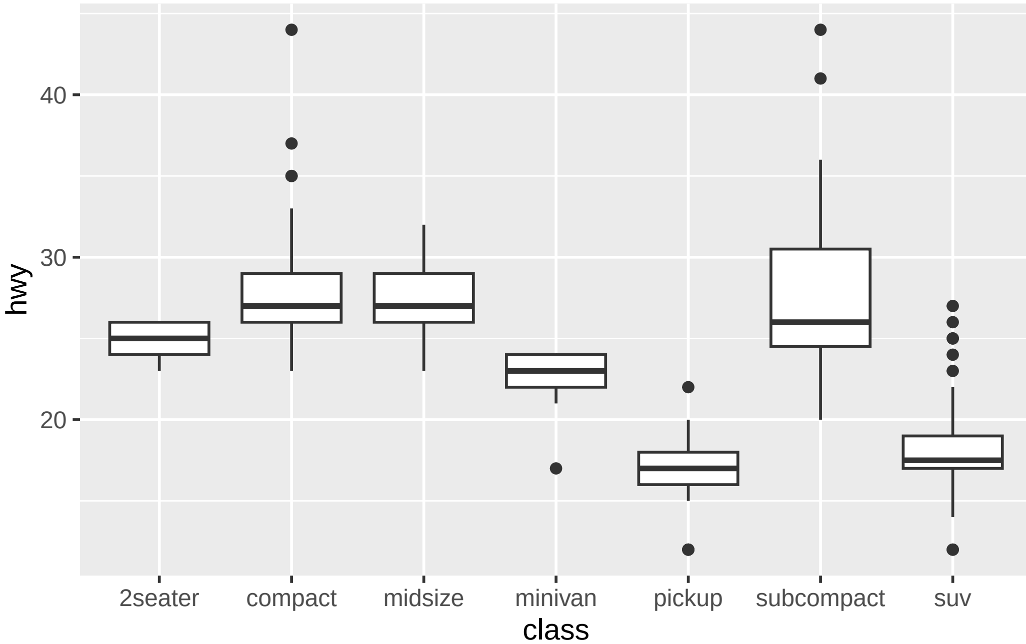
```
ggplot(data = diamonds,
       aes(x = price, y = ..density.., color = cut)) +
  geom_freqpoly()
```

```
Warning: The dot-dot notation (`..density..`) was deprecated in ggplot2 3.4.0.
i Please use `after_stat(density)` instead.
```



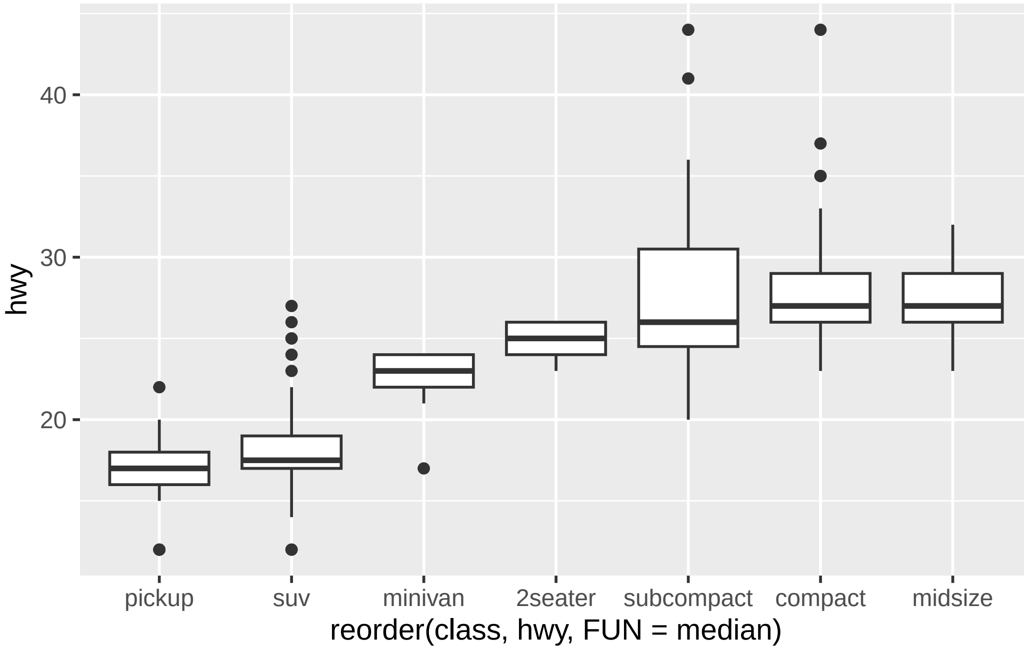
Using boxplots to analyze too…

```
ggplot(data = mpg, aes(x = class, y = hwy)) +
  geom_boxplot()
```



Reorder it os is more clean...

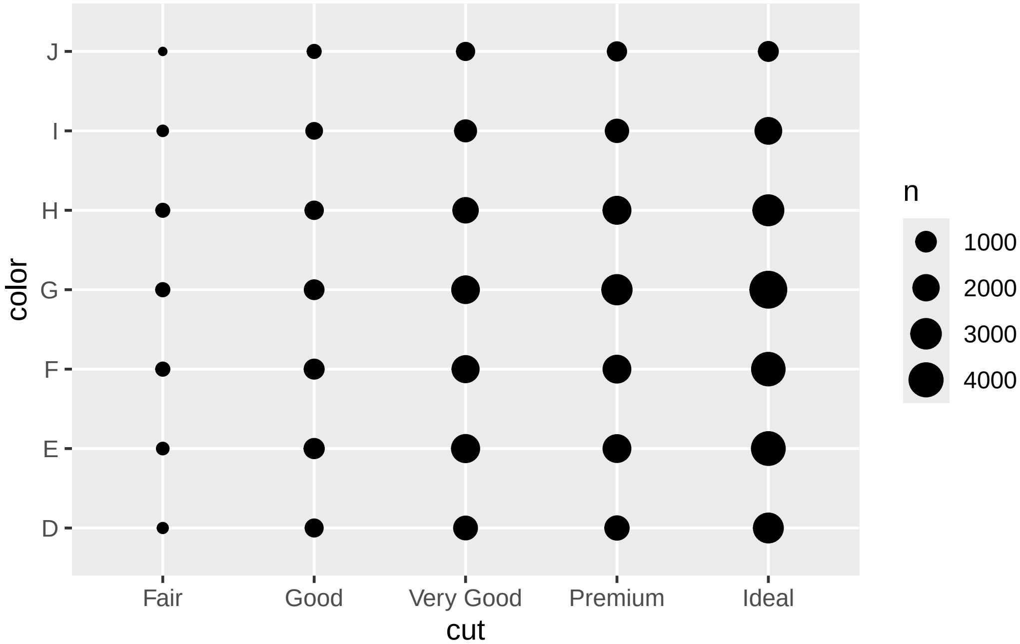
```
ggplot(data = mpg,
       aes(x = reorder(class, hwy, FUN = median),
           y = hwy)) +
  geom_boxplot()
```



Covariance: Categorical - Categorical

We have seen bar plots, but more ways are possible...

```
ggplot(diamonds, aes(x=cut, y=color)) +
  geom_count()
```



Let's do it with code using **dplyr** (two ways)

```
diamonds %>%
  group_by(color, cut) %>%
  summarise(n = n())
```

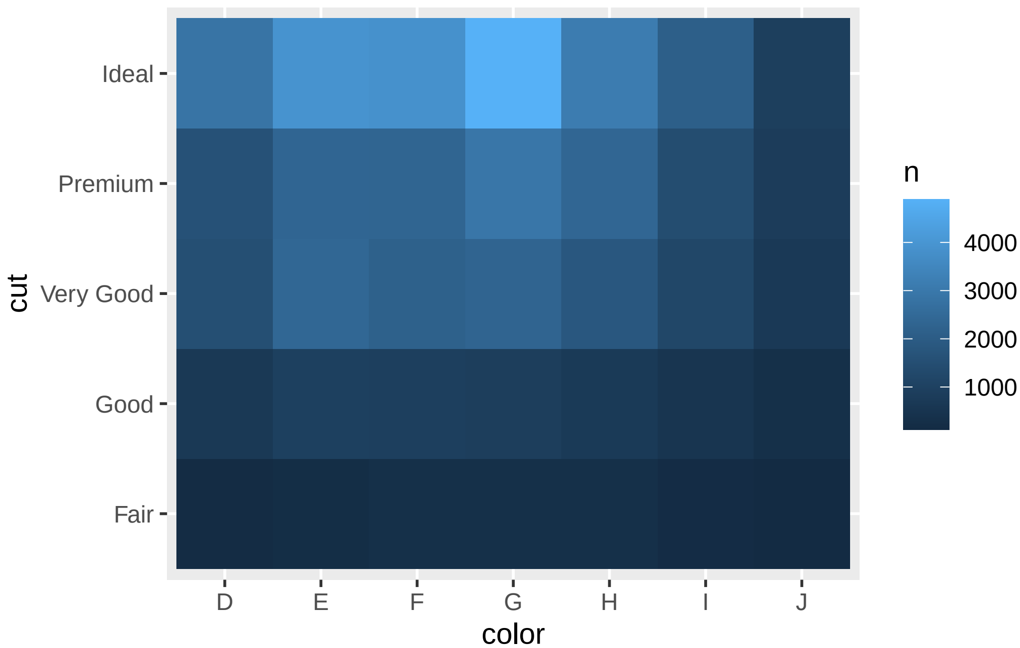
```
`summarise()` has regrouped the output.
i Summaries were computed grouped by color and cut.
i Output is grouped by color.
i Use `summarise(.groups = "drop_last")` to silence this message.
i Use `summarise(.by = c(color, cut))` for per-operation grouping
  (`?dplyr::dplyr_by`) instead.
```

```
# A tibble: 35 x 3
# Groups:   color [7]
   color cut           n
   <ord> <ord>     <int>
 1 D     Fair        163
 2 D     Good        662
 3 D     Very Good  1513
 4 D     Premium    1603
 5 D     Ideal      2834
 6 E     Fair        224
```

```
 7 E      Good         933
 8 E      Very Good   2400
 9 E      Premium     2337
10 E      Ideal       3903
# i 25 more rows
```

More visual approach, using color mapping…

```r
diamonds %>%
  count(color, cut) %>%
  ggplot(aes(x = color, y = cut, fill = n)) +
  geom_tile()
```

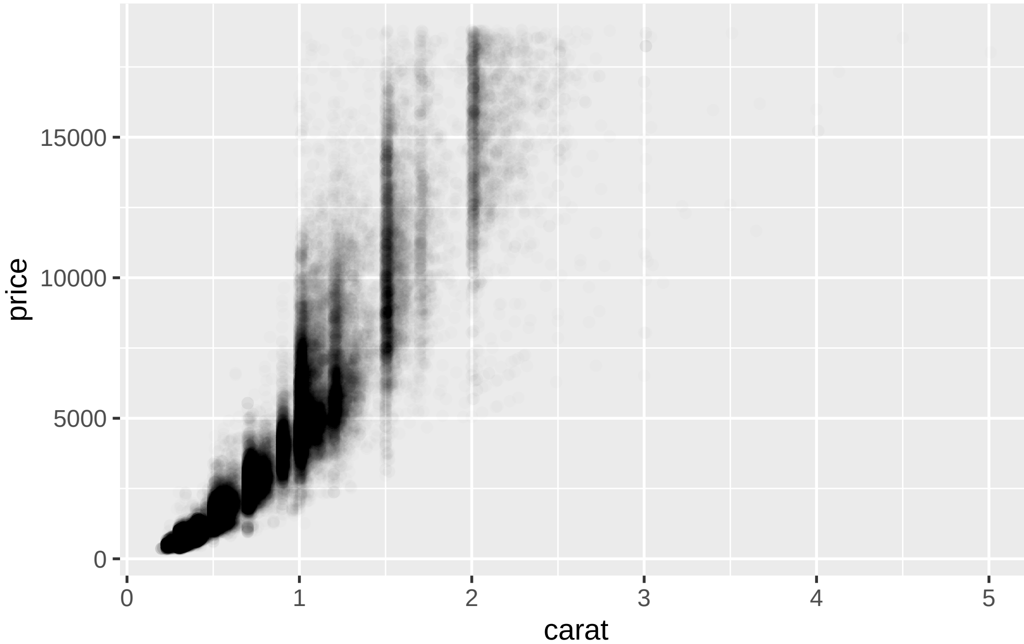


Covariance: Continuous - Continuous

Normally we will use an scatter plot with `geom_point()`, but we can find some problems when there are too many data points in big datasets, they will overalp each other.

In these cases we will need to use `alpha` aesthetic:

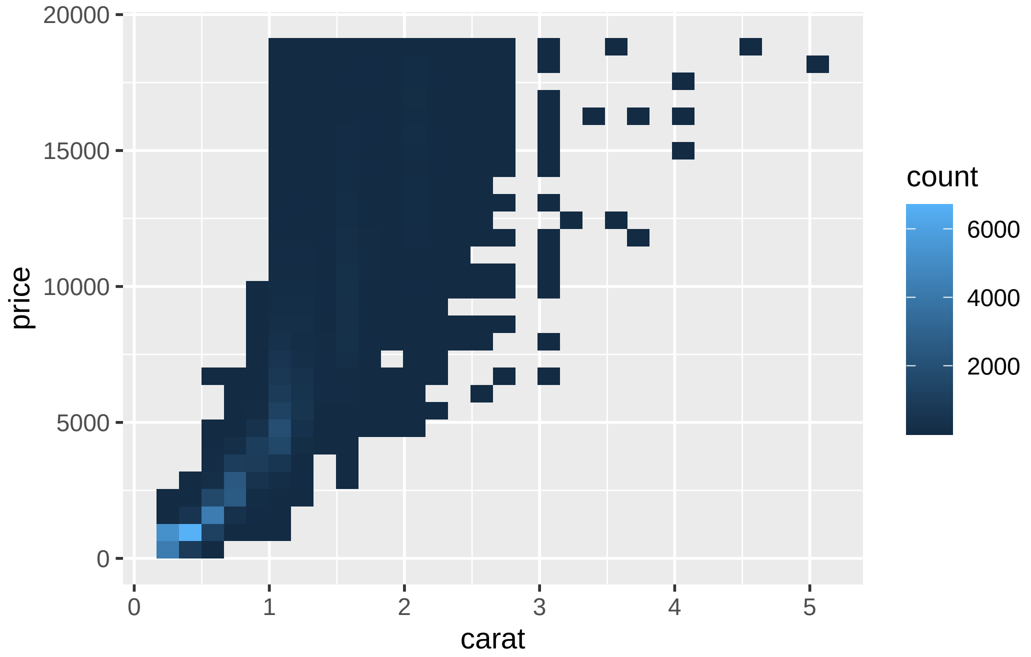
```r
ggplot(diamonds, aes(x = carat, y = price)) +
  geom_point(alpha = 0.01)
```



Another approach...

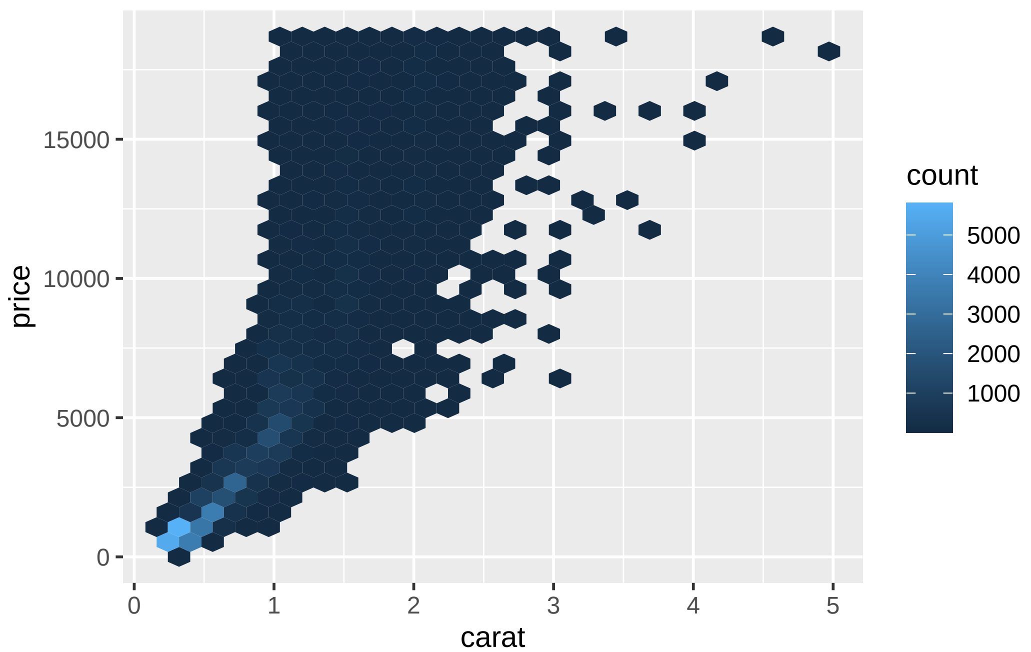
```
ggplot(diamonds, aes(x = carat, y = price)) +
  geom_bin2d()
```

```
`stat_bin2d()` using `bins = 30`. Pick better value `binwidth`.
```



Another approach... but better

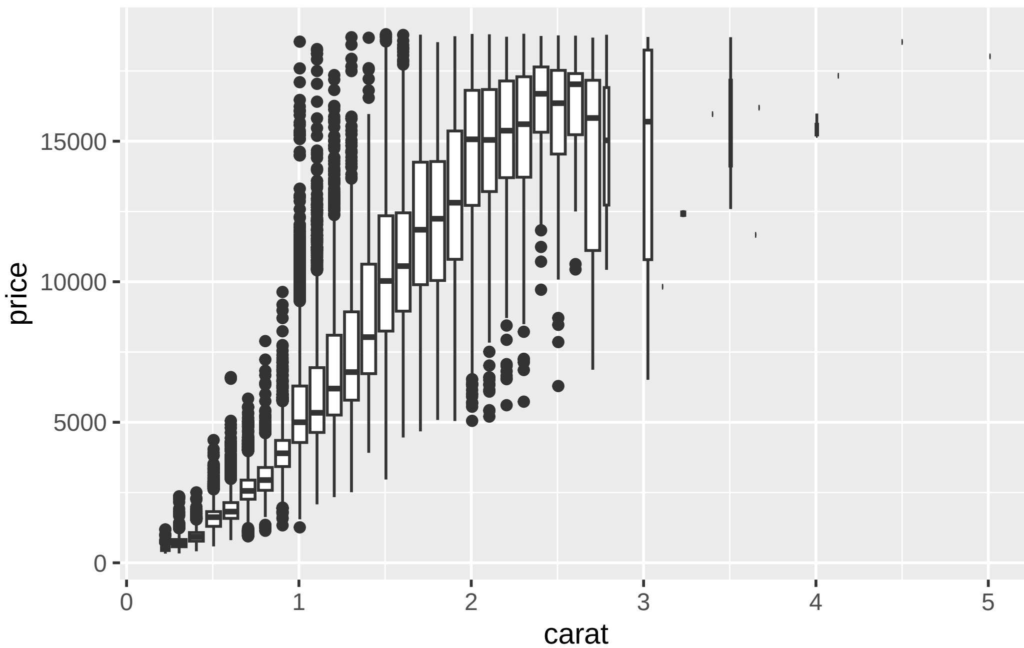
```
# install.packages("hexbin")
ggplot(diamonds, aes(x = carat, y = price)) +
  geom_hex()
```



Box plots work here too:

```
ggplot(diamonds, aes(x = carat, y = price)) +
  geom_boxplot(aes(group = cut_width(carat, 0.1)))
```

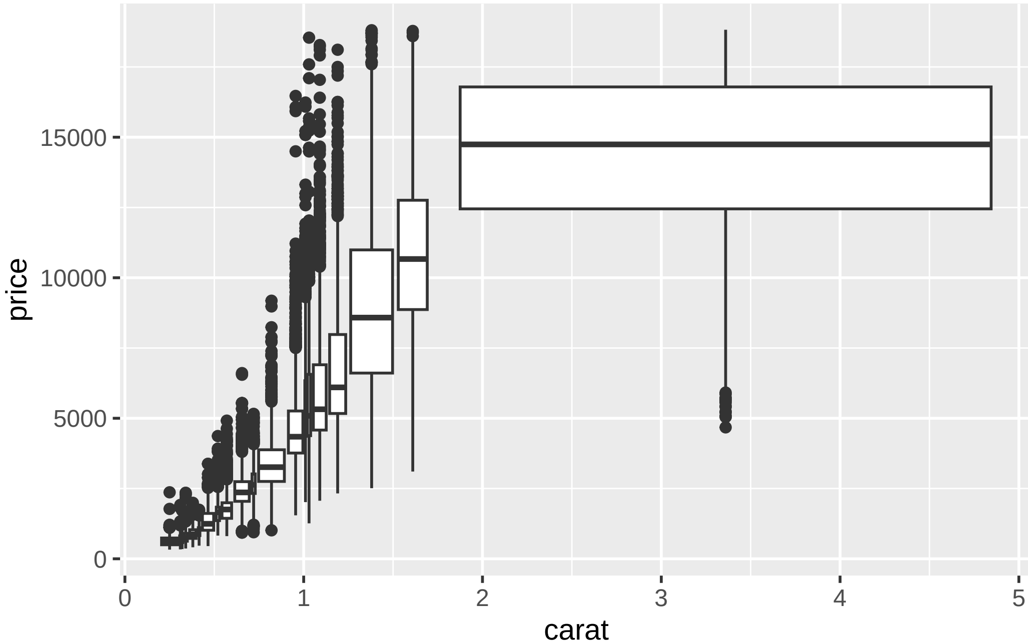
```
Warning: Orientation is not uniquely specified when both the x and y aesthetics are
continuous. Picking default orientation 'x'.
```



Same number of point for each bin:

```
ggplot(diamonds, aes(x = carat, y = price)) +
  geom_boxplot(aes(group = cut_number(carat, 20)))
```

```
Warning: Orientation is not uniquely specified when both the x and y aesthetics are
continuous. Picking default orientation 'x'.
```



Patterns and models

Patterns reveal hints regarding relations between variables. When we see one we must analyze…

- Can be random?
- How we can describe the relation that shows?
- Is relevant?
- Are there any other variables affected by this relation?
- This relation changes if we analyze subgroups of data?

Models can be used for extract patterns of the data, like eliminating the effect of a variable and study the remaining effect.